

 SENAI – Serviço Nacional de Aprendizagem Industrial CFP- Cândido Athayde (NFP-Raimundo de Sousa Lima) Atividade	3º Ano (A) Comp. Curricular: Professor(a): Cleiton Data: ____/____/____
Aluno (a): Pedro Paulo Lima Sousa	Nº: 25

Parte 1: Otimização de Código e Legibilidade

1. Por que a legibilidade é considerada um dos pilares da otimização de código?
Porque um código fácil de entender facilita alterações, correções e revisões.
2. Explique como o uso de nomes significativos para variáveis e funções reduz a necessidade de documentação externa.
Porque o próprio código já explica sua função.
3. Como a divisão de funções extensas em blocos menores facilita a manutenção e a depuração de erros?
Porque cada parte fica responsável por uma tarefa específica.
4. Defina o conceito de código morto e explique por que ele deve ser removido do projeto.
Código morto é o trecho que existe no projeto, mas nunca é executado ou utilizado.
5. Qual é o impacto da eliminação de trechos de código não utilizados no tamanho final do arquivo executável?
Eliminar o código não utilizado, não vai alterar no tamanho final do arquivo.
6. De que forma o código modularizado contribui para a sustentabilidade de um software a longo prazo?
Porque divide o sistema em partes menores, independentes e mais fáceis de entender. Assim, quando surge a necessidade de corrigir um erro, atualizar uma função ou adicionar uma nova parte, não é preciso mexer em todo o programa. Isso torna a manutenção mais simples, reduz a chance de gerar novos problemas e facilita o trabalho em equipe.

Parte 2: Desempenho e Algoritmos

7. O que a análise de complexidade algorítmica busca medir em relação ao crescimento do volume de dados?
A análise de complexidade algorítmica mede como o tempo e o uso de memória crescem conforme a quantidade de dados aumenta.
8. Por que as estruturas de repetição como laços de loop costumam ser o foco principal das otimizações de desempenho?
Laços de repetição costumam ser o foco das otimizações porque executam muitas vezes as mesmas instruções.
9. Descreva a técnica de minimização de cálculos repetitivos dentro de estruturas de laço.

A técnica de minimizar cálculos repetitivos consiste em calcular uma vez o que pode ser reaproveitado várias vezes.

10. Como a escolha de um algoritmo de busca eficiente afeta a escalabilidade de um sistema de software?

Um algoritmo de busca eficiente melhora a escalabilidade porque encontra dados com menos operações.

11. Explique como o acesso linear aos dados na memória RAM pode otimizar o uso do cache do processador.

O acesso linear à memória ajuda o cache do processador porque aproveita melhor a proximidade física dos dados.

12. Qual o risco de escolher algoritmos ineficientes ao lidar com bases de dados que crescem exponencialmente?

O risco de usar algoritmos ineficientes em bases que crescem muito é o sistema ficar lento ou até inviável.

13. O que significa realizar uma otimização voltada para a organização de vetores e matrizes na memória?

Otimizar a organização de vetores e matrizes na memória significa armazenar os dados de forma que o acesso seja mais rápido e eficiente.

Parte 3: Rotinas e Mecanismos de Callback

14. Defina o que são sub-rotinas e como elas auxiliam na reutilização de blocos lógicos dentro de um programa.

Sub-rotinas são blocos de código que executam uma tarefa específica e podem ser chamados quando necessário.

15. Como a centralização de rotinas comuns permite realizar otimizações globais em todo o sistema de forma mais simples?

Centralizar rotinas comuns facilita otimizações globais porque uma única melhoria pode beneficiar várias partes do sistema.

16. Explique o funcionamento básico de um mecanismo de callback e em que situações ele costuma ser aplicado.

Callback é uma função passada para outra função para ser executada depois, geralmente quando um evento termina ou acontece.

17. Qual é a principal diferença entre a execução de uma função síncrona e uma função que utiliza mecanismos de retorno de chamada?

A função síncrona executa e só depois libera a próxima instrução; a função com callback pode chamar outra função quando terminar.

18. Como o gerenciamento de fluxos assíncronos evita que a interface do usuário trave durante tarefas de longa duração?
O gerenciamento de fluxos assíncronos evita que a interface trave porque tarefas longas são tratadas sem bloquear o restante do programa.
19. Por que os callbacks são fundamentais para o desenvolvimento de sistemas baseados em eventos e interfaces reativas?
Callbacks são fundamentais em sistemas baseados em eventos e interfaces reativas porque permitem responder a ações em tempo real.
20. O que significa dizer que um sistema possui redundância de código e como as rotinas ajudam a solucionar este problema?
Redundância de código é quando a mesma lógica aparece repetida em vários lugares do programa. As rotinas resolvem isso concentrando essa lógica em um único ponto.